4) Bastion-to-Node SSH Access Configuration

For Kubespray (Ansible) to connect from the Bastion server to each K8s node and perform tasks with root privileges, passwordless SSH access configuration is required.

In this step, we will create and execute a setup-ssh.sh script. This script generates an SSH public key on the Bastion server and copies it to both the REMOTE_USER account and the root account on all K8s nodes (Masters and Workers).

4.1. Creating the SSH Connection Script

If the prerequisites are not met, you may need to generate keys and copy them to remote nodes manually.

Create the setup-ssh.sh script file in the kubespray-2.29.0 directory on the Bastion server.

```
$ vim setup-ssh.sh
```

Copy the script content below into the file.

```
#!/bin/bash
# ------------------------------------------------------------------
# [All-in-One] Bastion Key Generation, Copy to Remote Nodes, and Root Privilege Escalation Script
#
# [Prerequisites]
# 1. If 'sshpass' is installed on the Bastion server running this script,
#    it asks for the password only once; otherwise, 'ssh-copy-id' asks every time.
# 2. The REMOTE_USER account on all remote nodes must be able to sudo with 'NOPASSWD'.
# ------------------------------------------------------------------

# [Modification Required] Internal IP addresses of K8s nodes
HOSTS=(
  "***.***.***.***"
  "***.***.***.***"
  "***.***.***.***"
  "***.***.***.***"
  "***.***.***.***"
  "***.***.***.***"
```

```
)

# [Modification Required] sudo-capable account to connect to nodes
REMOTE_USER="*****"

# --- 1. Generate Bastion SSH Key (if missing) ---
BASTION_KEY_FILE=~/.ssh/id_rsa
BASTION_PUB_KEY_FILE=${BASTION_KEY_FILE}.pub

if [ ! -f "$BASTION_KEY_FILE" ]; then
    echo "### SSH key not found. Generating new key..."
    # -N "" : Generate key without passphrase
    ssh-keygen -t rsa -b 4096 -N "" -f "$BASTION_KEY_FILE"
    echo "### Key generation complete."
else
    echo "### Using existing SSH key ($BASTION_KEY_FILE)."
fi

# Store Bastion public key content in a variable
BASTION_PUB_KEY_CONTENT=$(cat "$BASTION_PUB_KEY_FILE")
if [ -z "$BASTION_PUB_KEY_CONTENT" ]; then
    echo "### Error: Cannot read Bastion public key."
    exit 1
fi

echo "============================================================"
echo "### Phase 1: Starting SSH key copy to each node as ${REMOTE_USER}."
echo "### You may need to enter the password for ${REMOTE_USER} for each host."
echo "============================================================"
echo ""

# --- 2. (Phase 1) & 3. (Phase 2) ---
for host in "${HOSTS[@]}"; do
    echo "--- [${host}] Operation Start ---"

    # --- Phase 1: Execute ssh-copy-id ---
    echo "Attempting to copy key to ${REMOTE_USER}@${host}..."
    ssh-copy-id -i "$BASTION_PUB_KEY_FILE" "${REMOTE_USER}@${host}"

    # Check success of ssh-copy-id
    if [ $? -ne 0 ]; then
        echo "### Error: Key copy to [${host}] failed. Check password."
        echo "### Skipping this host."
        echo "--- [${host}] Operation Failed ---"
        echo ""
        continue
    fi

    echo "Key copied to ${REMOTE_USER}. Proceeding to root setup..."

    # --- Phase 2: Configure root key using sudo on remote ---
    # Pass BASTION_PUB_KEY_CONTENT variable for use in remote shell
    ssh -t "${REMOTE_USER}@${host}" "
        # Quote to prevent splitting of spaces/special characters
        BASTION_KEY=\"${BASTION_PUB_KEY_CONTENT}\"
```

```
    echo 'Creating Root SSH directory and setting permissions...'
    sudo mkdir -p /root/.ssh
    sudo chmod 700 /root/.ssh

    echo 'Adding Bastion key to authorized_keys...'
    # Safely add only the Bastion key to 'root' authorized_keys.
    echo \\$BASTION_KEY | sudo tee -a /root/.ssh/authorized_keys > /dev/null

    echo 'Setting key file permissions and ownership...'
    sudo chmod 600 /root/.ssh/authorized_keys
    sudo chown -R root:root /root/.ssh

    echo 'Restoring SELinux context...'
    sudo restorecon -R -v /root/.ssh || true

    echo '[${host}] root setup complete.'
  "

  echo "--- [${host}] Operation Complete ---"
  echo ""
done

echo "============================================================"
echo "All operations completed."
echo "You can now attempt passwordless connection via 'ssh root@<Host_IP>'."
echo "============================================================"
```

4.2. Modifying the Script: Node IPs and Remote User

Open the file again using `vim setup-ssh.sh` and modify the two `[Modification Required]` sections to match your actual environment.

- `HOSTS=( ... )`: List all K8s Master and Worker node IP addresses to be added to `inventory.ini`, separated by spaces within the quotes.
- `REMOTE_USER="*****"`: Enter the name of the account that is pre-created on all K8s nodes and capable of executing sudo without a password (NOPASSWD).

Modification Example:

4.3. Running the Script

After saving the modified script, grant execution permissions to the file and run it.

```
 # Grant execution permission (x) to the script file.
$ sudo chmod +x ./setup-ssh.sh

# Run the script.
$ ./setup-ssh.sh
```

[Execution Note] When the script runs (Phase 1), you will be prompted multiple times for the `REMOTE_USER` password to connect to each server in the `HOSTS` list. Once you enter the correct password, the Bastion's public key will be copied to the `REMOTE_USER`, and then automatically copied to the `root` account.